Challenge Binding in FRESCO SPDZ2k: A Message-Level Case Study

Mingzhao Huang
Shanghai Jiao Tong University
huangmingzhao@sjtu.edu.cn

2026-11-03

AI contribution designation: Predominantly AI-generated work.

Abstract

FRESCO's SPDZ2k suite derives its batched MAC-check coefficients from a setup-time public seed, making the stream predictable before opening shares are sent. We examine this implementation path at a pinned revision. A source-level derivation accounts for low-limb carries and high-bit corrections and shows that the value-share shifts $e = (r_j, -r_i)$ preserve the check residual while retaining the original MAC shares. A message-level harness executes this deviation against unmodified honest opening code. The recorded sum/product outputs violate a relation required for every possible input of the corrupt party, and the test framework records normal completion. The retained evidence includes two-, three-, and four-party executions with dummy preprocessing. The predictable-seed attack template appears in *Rushing at SPDZ*, and FRESCO's issue tracker recorded the per-check randomness requirement in 2018; our contribution is the SPDZ2k public-seed path, its explicit kernel witness, and the implementation evidence. We also give a conditional single-opening analysis and assess a four-file candidate mitigation that rejects seven recorded tampering scenarios while preserving four honest baselines. Audit tools and descriptive corpora provide context. The proof concerns the specified value-share deviation; unrestricted malicious behavior, production preprocessing, and a complete mitigation argument remain outside the established result.

1 Introduction

FRESCO's SPDZ2k suite uses random coefficients to combine authenticated openings into one MAC check [16, 41]. At the revision studied here, those coefficients come from a deterministic random-bit generator (DRBG) initialized during setup. The seed contributions are opened publicly. Each participant can therefore reconstruct the coefficient stream before sending the shares that will be checked. A corrupt participant can use this knowledge to change two openings while preserving their weighted sum.

The algebra is simple. For coefficients r_i and r_j, the errors $e_i = r_j$ and $e_j = -r_i$ satisfy $r_i e_i + r_j e_j = 0$ in the arithmetic ring. Turning this cancellation into an implementation result requires more than solving that equation. The participant must control the relevant state and messages, use the correct coefficient positions, preserve the MAC shares, and survive the implementation's correction and checking paths. Our case study follows these steps through a message-level harness and retained execution logs. In the sum/product program, honest party 1 supplies 20 and corrupt party 2 supplies 30. The accepted tampered outputs are inconsistent with every possible input of party 2 (Section 4.3).

The underlying attack template is established prior work. *Rushing at SPDZ* [36], Section 6.2.1 and Appendix A, describes how predictable batch coefficients allow a corrupt party to

select incorrect openings with an unchanged linear combination. It also identifies a FRESCO commitment-replay path to a predictable seed. FRESCO's issue #324 [20], opened in 2018, separately records that reusing one joint DRBG across MAC checks is insecure. This paper examines the SPDZ2k setup-time public-seed path and supplies an explicit kernel construction and execution evidence for that path. Section 5 compares the mechanisms and their overlap.

We organize the audit around two implementation questions. Transcript coverage (C1) asks whether a derived challenge binds the relevant pre-challenge choices, including their encoding and protocol context. Randomness ordering (C2) asks whether a sampled challenge retains the required conditional randomness when those choices become fixed. Sampling after a binding boundary is a sufficient implementation pattern for C2; earlier generation can also be sound when the challenge remains hidden. These questions help locate implementation obligations, but each check still needs its own soundness argument.

The paper makes three contributions:

1. A FRESCO SPDZ2k case study connecting a predictable coefficient stream to an explicit two-opening kernel vector and recorded incorrect outputs. The message-level harness modifies only the corrupt party's opening behavior; honest parties use the original opening protocol. The source-level argument treats the specified value-share deviation and the coupling conditions stated in Appendix A.
2. An evidence account that distinguishes kernel-vector runs, a separate message-level series, negative controls, and the earlier eleven-case suite. It includes a single-opening analysis under explicit key and coefficient assumptions, a candidate mitigation, and tools for synthesizing vectors and emitting executable tests.
3. Audit context from an annotated transcript monitor and a heuristic source analyzer, with 19 zero-knowledge implementation entries and ten selected MPC scan targets. Their counting units and validation limits are stated separately from the FRESCO execution results.

The implementation runs use dummy preprocessing and concern FRESCO commit `f8383e6aead6b44d44b798553aaf0bcaccd81dab`. The available logs support the recorded executions; they do not measure attack prevalence or deployment impact. The MASCOT probe and the emp-agmpc model remain secondary check-level observations. Section 2 states the model, Section 3 introduces the audit context, and Section 4 presents the FRESCO evidence. Sections 5 and 6 discuss prior work and reproducibility. Appendices A–C give the source derivation, core reproduction recipe, and descriptive audit material.

2 Model and conditional guarantees

2.1 The checked choice and the commitment boundary

Let $V : X \times R \to \{0, 1\}$ be a verification predicate, and let $X^* \subseteq X$ be the set of correct checked choices. The check accepts when $V(x, r) = 1$; a successful forgery has $x \notin X^*$. Here x contains the choices that must be fixed for the particular checking argument. It need not contain the entire proof or protocol state. Responses that legitimately depend on a challenge require separate treatment.

The **commitment boundary** is the point after which the adversary cannot change these choices without violating an independently enforced constraint. Sending a message, storing a local value, and making a binding commitment can impose different constraints. Let v be the adversary's view at this boundary, including public parameters, its local randomness, received messages, and accessible generator state.

For sampled challenges, **C2** asks whether the challenge distribution, conditioned on v and the fixed choice x, satisfies the premises of the check's soundness argument. Sampling independent

coins after a binding boundary is one sufficient pattern. Earlier generation can also be suitable if the challenge remains hidden under the protocol's assumptions. Conversely, creating a generator after the boundary does not help when its seed already determines a stream known to the adversary.

For derived challenges $r = H(\tau(x))$, **C1** asks whether the encoded transcript binds the required pre-challenge choices, up to equivalences justified by the protocol. Syntactic dependence on every field is insufficient: an ambiguous encoding can map distinct choices to the same transcript. Required fields, encoding, domain context, and alternative binding through protocol constraints must be considered together. The monitor in Section 3 checks an annotated approximation to this requirement.

The companion ZK manuscript separates coverage (B1), encoding (B2), and domain separation (B3). Here C1 names the transcript-binding question as a whole; the monitor evaluates its annotated coverage component.

2.2 Sampled and derived challenges require different bounds

Proposition 1 (fixed-message sampling). Suppose that, for every reachable boundary view v and incorrect choice x compatible with that view, an independent draw $r \leftarrow R$ satisfies $\Pr[V(x, r) = 1 \mid v, x] \leq \varepsilon$. If the adversary fixes x before this draw, its forgery probability is at most ε.

Proof. Condition on the view and the fixed incorrect choice, apply the assumed bound, and average over their joint distribution. Chronological order alone does not establish the conditional premise.

Proposition 2 (a restricted derived-check model). Let H be an ideal random oracle with uniform range R, and let τ be injective on the modeled choice space. Assume that X, R, X^*, τ, and V are fixed independently of H, and that $\Pr_{r \leftarrow R}[V(x, r) = 1] \leq \varepsilon$ for every $x \notin X^*$. A classical adversary with auxiliary information independent of H, making at most q oracle queries and then outputting one x, has forgery probability at most $\min\{1, (q + 1)\varepsilon\}$. The query budget includes queries used to construct its view or perform preprocessing.

Proof. Consider the first query to each distinct oracle input in the image of τ. Injectivity determines a unique candidate x before the fresh oracle answer is sampled. For an incorrect candidate, acceptance has probability at most ε. A union bound covers at most q queried candidates. If the final output's transcript was not queried, verification uses one further fresh answer and contributes at most one further ε term. Repeated queries provide no fresh trials.

These are standard bounds, stated to delimit the model rather than as new results [35, 5]. Proposition 2 concerns a fixed predicate and an injective encoding. General Fiat-Shamir proofs include challenge-dependent responses and require their own security arguments [5]. A dependency-set check does not establish these premises, and collision resistance alone does not make a derived challenge independent of an adaptively chosen input.

2.3 Failure patterns are conditional

Learning r before fixing x permits the adversary to choose x as a function of the challenge. Whether it can efficiently find a reachable incorrect choice satisfying $V(x, r) = 1$ depends on the check. Public coefficients therefore warrant investigation but do not, by themselves, prove an efficient attack.

An omitted transcript component likewise yields a forgery only if a reachable incorrect completion satisfies the verification relation. Redundant absorption, external commitments, fixed protocol schedules, or constraint-level binding can prevent exploitation. A coverage warning records a modeling or implementation question until this reachability is established.

A check-level bypass compromises protocol integrity only when the deviation reaches a security-relevant result and satisfies the remaining checks. A perturbation may leave the final output unchanged or violate another invariant. Section 4 therefore establishes effects through case-specific code paths and recorded executions; we claim no general composition theorem.

2.4 Error spaces are part of the model

A linear cancellation construction seeks admissible errors satisfying $\sum_i r_i e_i = 0$ in the implementation's arithmetic domain. Its relevance depends on the actual acceptance predicate and adversary control. Restricting a solver to nonnegative low-bit errors can describe one experiment without describing all state changes available to a corrupt participant.

For the SPDZ2k opening path derived in Appendix A, zero weighted error is a sufficient, key-independent cancellation condition. It need not characterize every accepted deviation: arithmetic is over a power-of-two ring, the MAC key can be a zero divisor, and high-bit correction terms affect the residual. Applying a fixed-message bound or interpreting a solver's success frequency requires a justified acceptance predicate and error space. Appendix A derives the source-level identity for this path and states the distributional assumptions behind the single-opening calculation.

2.5 The restricted opening adversary

Definition 1 (opening-corruption adversary). Consider one sequential SPDZ2k computation with n parties and one statically corrupted party P^*, chosen before execution. The adversary $\mathcal{A}_{\text{open}}$ controls that party's local state and outgoing messages. It follows the protocol except that, at selected openings, it may add a chosen ring element e to its own value share, retain the original MAC share, and use the shifted share in the opening and subsequent correction computations. It may choose e from the public coefficient stream, the known batch schedule, and its local view. It has no access to the full MAC key α or honest parties' private state, and does not modify honest code, preprocessing, coin-toss commitments, or the network schedule. For the authenticated values and program under evaluation, it wins if an honest party completes with an incorrect opening or final result without aborting.

These deviations are allowed by a malicious model with the same static corruption and execution assumptions. A reachable successful execution therefore demonstrates an integrity failure in that setting. The present analysis covers only the specified opening deviations; adaptive corruption, additional corrupt parties, concurrent sessions, and other deviations require separate analysis.

Proposition 3 (cancellation on the specified opening path). At the pinned 64/128-bit implementation, couple a baseline and a modified check as in Appendix A.1: preserve the ordered coefficients, key and MAC shares, and the additional random-mask shares, and use the shifted value shares consistently in both opening messages and correction computations. Assume that the baseline residual is zero and the broadcast and commitment stages complete normally. If the corrupt party's errors satisfy

$$\sum_i r_i e_i \equiv 0 \pmod{2^{128}},$$

then the modified check also has zero residual and accepts. A nonzero $e_i \bmod 2^{64}$ changes the corresponding opened low bits.

Proof. The source-level identity derived in Appendix A.1 is $\Delta\zeta = \alpha \sum_i r_i e_i$. The stated congruence makes the change zero, so the zero baseline residual is preserved. The low-bit conclusion follows from the change in the consistently transmitted low shares.

For a batch with two nonzero 64-bit coefficients, the errors $e_i = r_j$ and $e_j = -r_i$ give the required congruence and change both low parts without an inverse or a key-parity assumption. The single-opening boundary depends on the coefficient and MAC key (Appendix A.2 and Section 4.4).

The proposition addresses the specified share-replacement path. It does not cover arbitrary malicious correction messages, changed supplier positions, or different batch schedules. Section 4 connects its conditions to the recorded message-level execution and the program's input/output relation.

3 Audit context

The C1/C2 distinction connects the FRESCO case to two complementary audit methods. The transcript monitor evaluates manually annotated dependency specifications with `absorb`, `squeeze`, and `check` events. Its coverage rule is set containment, supplemented by explicit model annotations. A passing verdict depends on the required set and its transcription; it does not prove encoding injectivity or cryptographic soundness.

OrderTool instead selects check-like source functions and applies heuristics for randomness provenance and message order. Its send/receive/opening patterns approximate a commitment boundary. Recognized public pools produce candidates for manual investigation, while apparent freshness or secret sharing is a source-pattern classification. The analyzer does not establish those security properties from program semantics.

The descriptive material comprises 19 zero-knowledge implementation entries across 18 projects, 26 specifications, and ten selected MPC scan targets. The ten retained parser reports contain 347 check-like records and two candidates. These units are distinct: specifications include historical repairs and synthetic fixtures, and parser records include constructors, accessors, and tests. The corpora support no population vulnerability estimate. Full entries, mechanism-level context, classification counts, and mutation-study limits appear in Appendix C.

The two MPC candidates lead to different outcomes. Section 4 presents implementation evidence for FRESCO. The emp-agmpc observation is a kernel calculation over an isolated compression relation; no corresponding protocol-level output forgery has been established. This distinction motivates the evidence chain used below: source-level control and scheduling, cancellation at the check, normal completion, and an output inconsistent with the program's functionality.

4 FRESCO SPDZ2k case study

4.1 Studied revision and coefficient source

The case study examines FRESCO commit `f8383e6aead6b44d44b798553aaf0bcaccd81dab` (2026-05-20), specifically `suite/spdz2k`. Values have 64 significant bits and a 128-bit arithmetic representation. We use k for the value width, K for the number of openings in a checked batch, and α for the MAC key.

The resource pool initializes a joint DRBG from publicly opened coin-toss contributions. Each participant can reconstruct its coefficient stream. `Spdz2kMacCheckComputation` obtains coefficients from `resourcePool.getRandomGenerator()`; each coefficient consumes `getHighBitLength()/8` bytes, or eight bytes in the tested configuration. Thus the coefficients occupy 64 bits, although the check operates in a 128-bit ring. They are predictable before the opening shares are sent, even though the implementation draws them later during the MAC check.

Each experiment uses one static corrupt participant, sequential execution, FRESCO's dummy preprocessing supplier, and party threads within one JVM. A generator callback records the

normal joint seed; it does not disclose an honest party's private randomness. The coefficient source is the resource-pool DRBG rather than the preprocessing supplier. Nevertheless, a different supplier or application can change the batch schedule. The experiments therefore establish behavior of the recorded online harness, without demonstrating a production-preprocessing or deployment attack.

4.2 Cancellation and message-level reachability

For the coupled share-replacement path derived from the pinned source in Appendix A.1, the residual change is

$$\Delta\zeta = \alpha \sum_i r_i e_i \pmod{2^{128}}.$$

For two positions with nonzero 64-bit coefficients, choose $e_i = r_j$ and $e_j = -r_i$ in the ring. Their weighted sum is zero, independently of α, and both low parts change. This is a sufficient cancellation construction for that path. The source derivation includes the carries introduced by changing a full share; it leaves arbitrary malicious correction messages outside the analysis.

The earlier object-level probes replace only the corrupt participant's authenticated-value object. The message-level probe implements the corresponding deviation as that participant's own native opening protocol: it sends the least-significant share of a shifted authenticated element, preserves the MAC share, and stores the shifted element for the later check. Honest parties execute the original `Spdz2kOutputToAllProtocol`. The high-bit correction is computed from the state supplied by the corrupt participant, so it must be included in the residual analysis. No change to honest opening code is needed for this test.

The application takes input 20 from honest party 1 and input 30 from corrupt party 2, then opens their sum and product. Its coefficient stream has four relevant positions across two MAC checks. Multiplication first opens two intermediate values. Before the subsequent output batch, `Spdz2kRoundSynchronization.beforeBatch` sees pending values and a protocol implementing `RequiresMacCheck`, so it checks these two values using stream indices 0 and 1. After the two final outputs are opened, `finishedBatch` checks them with indices 2 and 3. The sequential producer/evaluator preserves this dependency order [16]. Both checks therefore have $K = 2$. The harness's `coefficients(4)` helper replays four stream positions and selects the pair at offset 2; its argument is not the size of a single MAC-check batch. The corrupt participant applies the kernel vector to that final pair.

4.3 Recorded executions and their interpretation

Table 1 separates the test families. The $K = 2$ counts of 30, four, and one refer to the designated kernel-vector runs at three, two, and four parties. The application probes attack the final $K = 2$ check at stream offset 2. Their message-level executions and ten-run repetition series are separate evidence; they are not a subset of the raw-opening counts. These selected executions demonstrate behavior, without estimating attack prevalence or success over arbitrary strategies. Table 1 retains the historical canonical groups; the new clean-checkout verification described in Section 6.2 is reported separately.

Table 1: Retained FRESCO evidence by test family.
Evidence paths in this table are relative to `code/artifact/kernel/kernel/` in the associated `05-challenge-binding` package. A blocked result records the observed outcome; the table does not infer additional rejection semantics from a test name.

Suite	Configuration	Retained evidence	Outcome
Kernel-vector tests	$K = 2$, three parties, 30 runs	`runs/unpatched/ku-0.log` through `runs/unpatched/ku-29.log`	Incorrect openings returned
Kernel-vector tests	$K = 2$, two parties, four runs	`runs/unpatched/k2p.log`, `runs/unpatched/k2p-0.log` through `runs/unpatched/k2p-2.log`	Incorrect openings returned
Kernel-vector tests	$K = 2$, four parties, one run	`runs/unpatched/k4p.log`	Incorrect openings returned
Kernel variants	$K = 3/5/8$, $1/1/6$ runs	`runs/unpatched/k35.log` (K=3/5 only), `runs/unpatched/kv-0.log` through `runs/unpatched/kv-5.log`	Incorrect openings returned
Non-kernel controls	$K = 1/2/8$, three configurations	`runs/unpatched/ctrl1.log` through `runs/unpatched/ctrl3.log`	Blocked result lines
Patched kernel tests	$K = 2/3/5/8$, $6/1/1/6$ runs	`runs/patched/kf-0.log` through `runs/patched/kf-5.log`, `runs/patched/kpk35.log` (K=3/5 only), `runs/patched/fk8-0.log` through `runs/patched/fk8-5.log`	Kernel deviations blocked
Object-level application probe	Final $K = 2$ check, stream offset 2, three parties	`appoutput/run.log`	Wrong sum and product returned
Message-level application probe	Final $K = 2$ check, offset 2, two, three, and four parties; ten further three-party repetitions	`messagelevel/run.log`, `messagelevel/repeat-runs.txt`, `messagelevel/run-multiparty.log`	Wrong outputs returned; recorded non-kernel controls blocked
Disclosed cancellation suite	Eleven cases, thirteen original-code suite runs	`disclosed/runs/fresco-final1.log` through `disclosed/runs/fresco-final5.log`, `disclosed/runs/stab-0.log` through `disclosed/runs/stab-7.log`	Eleven test cases complete without runner errors; two attack assertions have the limitations stated below
Patched disclosed suite	Four baselines and seven tampering scenarios	`disclosed/dk.alexandra.fresco.suite.spdz2k.protocols.computations.TestSpdz2kMacCheckCancellation.txt`	Four baselines complete; seven scenarios produce errors reporting MAC-check rejection

In the object-level application log, honest execution returns `[50, 600]`; the tampered execution returns `[4143675807337242761, 9821738699198359411]`. The initial message-level log records `[15699666240690473753, 11601669154819341260]` after tampering, alongside the honest result `[50, 600]` and a blocked non-kernel control. Different executions use different coefficient streams, so their incorrect outputs differ.

These outputs also exclude a different legitimate input from the corrupt party. With honest input fixed at 20, any corrupt input $b \in \mathbb{Z}_{2^{64}}$ must produce $S = 20 + b$ and $P = 20b$, hence

$$P - 20(S - 20) \equiv 0 \pmod{2^{64}}.$$

The recorded initial message-level pair instead gives residue `11202993594072244072`. The displayed object-level pair gives `735198847291711055`. Both are nonzero, so neither pair is consistent with any choice of the corrupt party's input. These are arithmetic checks of the retained outputs, not additional protocol executions.

The message-level repetition summary records ten fresh-seed executions, each with a normal tampered-result line containing wrong outputs, an honest `[50, 600]` result, and a blocked control. The multiparty log separately records normal tampered-result lines at two and four parties and another three-party execution. Their role is to demonstrate the deviation through the corrupt party's message interface under the harness assumptions.

The normal-return result lines are printed after `runTest` returns. At the pinned revision, `AbstractSpdz2kTest` delegates to `TestThreadRunner.run`, which waits for all party threads to terminate, propagates recorded setup/test/teardown exceptions, and throws on timeout [16]. A normal result line therefore supports completion without a recorded party exception. The probes store outputs in a shared static `lastOpened` slot rather than retaining a separately labeled result from each party. Honest-party equality follows from the message path: the corrupt native sends the same low share to every party, and the original honest opening native sums the same received share vector and returns its low part. This source argument and the normal-return logs jointly support the honest-output claim. The attack assertions alone remain weaker: their exception-catching wrapper can return with an output already stored before a late abort, and their assertions do not explicitly require normal completion.

The eleven-case disclosed suite uses the earlier nonnegative-box solver and configures two, three, and four parties, small and 100-value batches, computed products, different corrupt participants, and later checks with stream offsets. Its thirteen retained suite runs include five consecutive fresh-seed runs. Two test oracles limit what the pass counts establish: `attackComputedProducts` compares products with a helper that incorrectly generates sums, so honest products can satisfy its inequality assertion; `attackMultiCheck` has no output-change assertion. These two cases establish normal suite execution, but their passes do not independently establish product corruption or a changed output in a later check. The dedicated application logs and the functional relation above supply the main computation-level evidence.

On the patched tree, four baselines complete and all seven tampering scenarios terminate with `MaliciousException: Mac check failed`, as recorded in the text report cited in Table 1. The unchanged attack expectations produce seven runner errors, so this is rejection evidence, not a passing patched regression suite. The sanitized XML companion contains unescaped angle-bracket placeholders and is not valid machine-readable XML; the table therefore cites the readable text report. All executions are author-run, including the fresh-clone record.

4.4 The single-opening boundary

The $K = 1$ analysis requires the MAC key as well as the coefficient. Under the specified opening path and its unchanged MAC/mask conditions, an accepted shift satisfies $\alpha re = 0$ in the 128-bit ring. For an odd key and a nonzero 64-bit coefficient, this condition forces the low 64 bits of e to vanish. With an even key, zero divisors can permit further accepted shifts; a zero coefficient makes every shift leave zero residual. Appendix A gives the conditions and distinguishes this analysis from an unrestricted malicious-security bound.

The exploratory single-opening test applies its high-bit edit only when the coefficient is even. Earlier logs omit that applicability flag, so an unchanged result may record a skipped edit. A later applicability control records edits in nine of twenty invocations and unchanged opened values in all twenty. These observations support no output-corruption claim and no edit-acceptance rate. They

also do not justify interpreting the older patched single-opening counts as a security result. The applicability log is retained at `code/artifact/kernel/kernel/runs/k1-applicability.log`.

4.5 Synthesis and source-audit tools

The tools in `code/artifact/synth/synth/` make several manual steps executable. Given a ring modulus, coefficients, and controllable positions, `synth.py` solves a cancellation congruence and verifies a zero weighted residual and changed low parts. The reports in `data/synth/synth-report-real.json` and `synth-report-k4.json` supply a three-coefficient parameter vector and a synthetic $K = 4$ instance. The first vector was transcribed in a historical fixed-seed debugging note as FRESCO output, but its original seed and raw log are not retained. We therefore use its supplied coefficients solely as algebraic inputs, without counting that report as an additional implementation execution. Their acceptance predicate is the supplied algebraic model, not a predicate recovered automatically from source.

The source audit marks `Spdz2kMacCheckComputation` as a candidate and classifies the mod-p suite's `SpdzMacCheckProtocol` and `SpdzRoundSynchronization` as using fresh coefficient generation (`data/synth/audit-report.json`). These textual classifications guide inspection; they do not establish the true commitment boundary or security of the fresh-generation path.

The JUnit emitter and execution driver regenerate a test, run it against a checkout, and record the result. Their application helper replays four coefficient-stream positions and tampers with the pair at offset 2; as in Section 4.2, this is the final $K = 2$ check, even though the recorded report calls the replay-length parameter `batch=4`. `data/synth/verify-report.json` reports Maven exit code zero, honest `[50, 600]`, and tampered `[7272616386388937259, 13889006554384388372]`. The binding script also reproduces the MASCOT probe's deltas `[6372, 19772]` in `data/synth/binding-report.json`. We use this script only to reproduce the recorded two-nonzero-mask prime-field instance. Its current implementation mishandles zero-mask and single-position ring cases, so its decision labels are not used as a complete attackability procedure. The synthesized vector and its directly verified residual, not auxiliary multiplicity labels, are the mathematical outputs used here. Controllable state, batch positions, and protocol steps remain manual configuration inputs; the tools do not discover a complete attack from an arbitrary implementation.

4.6 MASCOT: a separate conditional observation

The MASCOT probe concerns consistency between distributed shares and computed MACs [26]. A test-only hook makes the inputter share $v + e$ while the MAC computation uses v. In a two-party experiment with field modulus 65519, inputs `[11, 22]`, and fixed sampler seed `{1,2,3}`, the probe predicts the later mask stream and chooses $e = (\text{mask}_2, -\text{mask}_1)$. Both the input check and a subsequent open check return normally with `[6383, 19794]`.

This is a check-level observation under a predictable sampler. A corrupt inputter can already choose its own input; the observation concerns inconsistent authentication, not alteration of an honest participant's input. It does not establish that production MASCOT preprocessing exposes the same mask stream at the required time. The non-kernel control catches an arbitrary runtime exception, so it establishes that the control aborted without identifying the rejection mechanism from its assertion alone.

On the one tested integrated path, a two-party SPDZ computation with MASCOT preprocessing returns the honest value `[5]` in its baseline and aborts with `MaliciousException` when its input-mask batch is corrupted. Other consumption paths were not tested. The probe classes, hook installers, and logs are in `code/artifact/kernel/kernel/mascot/`. We retain these results as a conditional authentication-consistency observation and an aborting integrated experiment; they are not a second demonstrated protocol-level output forgery.

4.7 Candidate mitigation and timing limits

The four-file patch in `code/artifact/kernel/kernel/disclosed/fix.patch` derives a new coefficient generator during each MAC check, after the relevant openings. It invokes `CoinTossingComputation` with `HashBasedCommitmentSerializer` to generate the joint seed. The retained patched runs show rejection of the specified kernel deviations and the seven disclosed tampering scenarios.

A complete mitigation argument remains unfinished. It must establish the binding boundary for every message still under corrupt control, the coin-toss procedure's unpredictability and bias properties, replay and synchronization behavior, high-bit correction, and the actual ring-valued soundness bound. Rejection of strategies computed from the old stream does not supply that argument. We therefore describe the change as a candidate mitigation.

The timing directory, `code/artifact/kernel/kernel/timing/`, retains thirty labeled `seed_ns`/`check_ns` intervals per batch size. The instrumentation that produced those intervals is not retained with the measured revision, so their overlap and relationship to an unmodified baseline remain unverified. We report no percentage or deployment-overhead conclusion from these logs.

4.8 Disclosure record and related manuscripts

FRESCO issue #324 records the per-check coin-tossing rule publicly since 2018-12-10 [20]. The present case study examines this concern at the studied SPDZ2k revision.

The retained project record states that the reproduction, eleven-case suite, and four-file patch were emailed to the maintainers on 2026-10-20, citing issue #324. It records no reply as of 18:00 China Standard Time that day and schedules a follow-up for 2026-11-03. Later correspondence and release status are not established by this record.

The associated manuscripts use the same FRESCO tests, patch, and recorded runs and therefore share one experimental record. Their evidence is not counted as independent replication here.

5 Related work and positioning

5.1 Transcript binding

Fiat-Shamir [15] and its security analyses [35, 5] explain how transcript binding enters protocol-specific security arguments. Proposition 2 isolates a more restricted experiment with a fixed acceptance predicate. The weak/strong Fiat-Shamir distinction of Bernhard, Pereira and Warinschi [3], the implementation survey of Dao et al. [45], and the Frozen Heart disclosures [17] provide the main background for C1. Our 19 implementation entries overlap this earlier work and serve as descriptive audit examples. The transcript monitor checks manually supplied specifications of existing transcript code. CCC-Check [6] detects computation-constraint inconsistencies in ZK programs, while Zippel [47] develops a language and tooling for proof systems; these approaches address complementary implementation layers.

5.2 MPC implementation failures

SPDZ [40] and SPDZ2k [41] provide the authenticated-arithmetic setting for the case study. FRESCO's issue #324 [20] already records the need for fresh coin tossing at each batched MAC check and warns against reusing one DRBG. *Rushing at SPDZ* [36] studies implementation failures in MP-SPDZ [29], SCALE-MAMBA and FRESCO. Alongside its concurrency attacks,

Section 6.2.1 identifies a replay attack on FRESCO's `H(m||r)` commitments: replay can cancel honest seed contributions and make the joint seed predictable. Its Appendix A gives the batch-check attack template. An adversary that knows the seed can alter earlier openings and choose the final opening so that the checked linear combination is unchanged. The paper also notes the similar MAC-check structure in SPDZ2k.

The predictable-seed cancellation template is therefore established prior work. This case study follows the public-joint-seed path in `suite/spdz2k`, instantiates cancellation with the inverse-free ring vector $e = (r_j, -r_i)$, and records message-level tests in which unmodified honest parties accept incorrect openings. The single-opening analysis and candidate patch further delimit that implementation result. Table 2 identifies the documented routes and evidence; a complete comparison of every FRESCO code path remains open.

Table 2: Public guidance, the closest prior attack template, and the implementation evidence developed here.

Aspect	**Issue #324 [20]**	**Rushing at SPDZ [36]**	**This case study**
Randomness concern	Reuse of one DRBG across checks	Commitment replay makes the joint seed predictable	Public joint seed fixes the SPDZ2k coefficient stream before openings
Construction	Fresh-coin requirement	Choose a final opening to preserve the batch linear combination	Inverse-free two-opening kernel vector over the implementation ring
Evidence and scope	Implementation warning	FRESCO replay analysis and predictable-seed attack template	Recorded message-level openings, wrong sum/product outputs, single-opening boundary, and candidate patch

Other implementation failures concern different state or timing requirements. Off-Key [31] studies privacy failures caused by inconsistent MAC keys across batches, threads or process runs in MP-SPDZ, SCALE-MAMBA and FANNG-MPC. Work on delayed checks in Trident [44], Fantastic Four [13], SWIFT [43] and Quad [12] examines information released before an eventual abort. These results reinforce the need to trace the specific state, messages and security property affected by a check.

6 Limitations and reproducibility

6.1 Limits of the evidence

The corpora were selected for accessible, inspectable code. The zero-knowledge entries combine annotated specifications, source inspection, historical configurations and one entry outside the model. The MPC scan depends on chosen source roots, name filters, parser support and classification rules tuned during the investigation. Its counts describe retained parser output; they support neither prevalence estimates nor security conclusions for code without candidates.

The sampling and derived-check propositions state conditional guarantees. The sampling argument requires conditional fixed-choice soundness; the derived-check argument requires an ideal oracle, injective encoding and a fixed predicate. The tools do not establish these premises. Their conclusions also depend on the fidelity of the annotations and the source-level interpretation of commitment boundaries.

The FRESCO evidence covers one revision and local author-controlled runs with dummy preprocessing and party threads within one JVM. The message-level harness exercises two, three and four parties, and the retained computation demonstrates accepted wrong outputs. These are

existence demonstrations within the tested environment. Deployment reachability, unrestricted malicious behavior, attack rates and application overhead remain outside the evidence. The candidate patch rejects the recorded attacks but still requires a security argument for its coin tossing and remaining correction messages. MASCOT and emp-agmpc remain secondary check-level observations.

6.2 Local artifact and reproduction status

The local delivery package retains test code, reports and logs for inspection. Paths below are relative to `05-challenge-binding/`:

- `code/artifact/kernel/kernel/` contains the kernel-vector tests and retained runs, including the message-level, sum/product, disclosed-suite, MASCOT, patched and timing evidence.
- `code/artifact/synth/synth/` contains the synthesizer, binding analysis, source audit, JUnit emitter and driver, with recorded reports. Corresponding report copies are retained under `data/synth/`.
- `data/specs/` contains the 26 annotated transcript specifications; `data/ordertool/` contains the saved source-scan reports.

The code, recorded evidence, and this manuscript are available at `https://github.com/HMZ-fig/fresco-challenge-binding-artifact`. The `05-challenge-binding/` subtree preserves the relative paths used here and in Appendix B; the repository README separately indexes the legacy bundles. The dated verification records remain distinct from the historical runs.

The initial local audit on 2026-11-03 found that `code/artifact/run_all.py` requires source snapshots and some entry points absent from this package. The packaged `run_specs.py` also retains paths from the earlier directory layout. Correcting those paths in memory gave 26/26 expected specification verdicts. This specification check is separate from the implementation executions.

The author-requested local review then executed the Appendix B core recipe in a fresh checkout of the exact revision, with an isolated Maven repository, Apache Maven 3.9.9, and Temurin OpenJDK 21.0.11+10-LTS on Windows 11 amd64. The dependency build succeeded, and 28 inspected public source snapshots matched the checkout's Git blobs byte for byte. The unchanged message-level class completed five tests with zero failures or errors: the honest result was `[50, 600]`, the non-kernel control was blocked, and the two-, four-, and three-party tampered results all returned normally. Their functional residues were respectively `2688113812081450723`, `9173839401870698115`, and `17687041535159237539`, all nonzero. The unchanged disclosed suite completed eleven cases without errors on the original code. Applying only the supplied four-file patch produced seven `Mac check failed` errors and four completed baselines; the old oracle limitations remain as stated in Section 4.3.

These are new verification executions dated 2026-11-03, kept outside the historical canonical counts in Table 1. They are not independent laboratory replication. The logs, version output, valid new Surefire reports, test/patch hashes, and execution summary are retained under `data/verification-2026-11-03/`; `evidence-index.json` hashes the copied records and `run-manifest.json` identifies the precise inputs and command initialization differences. The historical logs were not overwritten. Appendix B supplies the current-layout recipe. Restoring the broader historical survey inputs and entry points remains a separate packaging task.

6.3 Remaining research questions

Appendix A derives the residual identity for the specified opening deviation. Extending the analysis requires characterizing additional malicious correction messages and other state changes beyond that path. The candidate mitigation also needs an argument covering commitment order, malicious influence on the joint seed, replay and generator synchronization. Independent checks of the transcript annotations and evaluation on a separately labeled source corpus would strengthen the audit tools. These tasks address the remaining gaps between a recorded failure, a faithful implementation model and a validated mitigation.

7 Conclusion

At the examined FRESCO revision, the public joint seed makes the SPDZ2k MAC-check coefficients predictable before the checked openings are fixed. In the recorded message-level harness, a corrupt party uses two share changes with $e = (r_j, -r_i)$ while preserving its MAC shares; unmodified honest parties accept incorrect openings. The recorded sum/product pair violates the relation required for every input of the corrupt party, with the honest input fixed at 20. The implementation contribution is this public-seed path, an inverse-free kernel witness and its execution through protocol messages, building on the predictable-seed attack template in prior work.

The candidate patch rejects the seven recorded tampering scenarios, and the synthesis and audit tools make the construction inspectable. The C1/C2 framework and descriptive corpora organize the surrounding audit questions. The demonstrated result is bounded by one revision, dummy preprocessing and local harness runs with party threads within one JVM; completing the malicious-state analysis, validating the mitigation and restoring the full historical investigation remain necessary next steps. Appendix B supplies the current-layout recipe for the core message-level test.

8 Assistance disclosure

Predominantly AI-generated work. AI systems performed most of the technical work and manuscript development under human direction. Their contributions included source analysis, proof development, experimental execution and verification, and manuscript preparation and revision. The human author primarily posed questions and provided guidance. The author remains responsible for understanding, verifying, and defending the claims, evidence, citations, and code, and for the integrity of the submitted work.

A Residual identity on the opening path

A.1 Source-level identity for the specified deviation

This derivation follows the `CompUInt128` opening and MAC-check path at the pinned FRESCO revision [16]. Set $B = 2^{64}$ and $M = B^2$. Unless another modulus is indicated, arithmetic is in $\mathbb{Z}_M$. For a canonical share $s \in \{0, \ldots, M-1\}$, define

$$\ell(s) = s \bmod B, \qquad h(s) = \lfloor s/B \rfloor, \qquad s = \ell(s) + Bh(s).$$

The source locations are under `suite/spdz2k/src/main/java/dk/alexandra/fresco/suite/spdz2k/`: the opening class is in `protocols/natives/`, the MAC-check class in `protocols/computations/`, and the limb and converter classes in `datatypes/`. The class and method names below identify the relevant operations at that revision.

Use i for opening positions and j for parties. Write s_{ij} and γ_{ij} for their value and MAC shares, α_j for key shares, and $\alpha = \sum_j \alpha_j$. Let (q_j, η_j) be the additional authenticated random-mask shares obtained inside `Spdz2kMacCheckComputation.buildComputation`.

Couple a baseline and a modified execution on the same ordered coefficient vector (r_i), key shares, checked MAC shares, and random-mask shares. Only the corrupt party c changes its value shares. It sends the low limbs of the shifted shares and retains those shifted full shares for correction. Opening messages and correction broadcasts are consistent across parties, so each party uses common opened values and a common correction. The following identity concerns this path; normal completion of the broadcast and commitment checks is a separate requirement.

The shared mask must come from the same supplier position in both executions. A changed batch schedule, a different mask, or a coefficient stream reset requires a separate comparison and is not absorbed into the share errors defined below.

`Spdz2kOutputToAllProtocol.evaluate` sends each low share as eight bytes. `CompUInt128` deserializes these bytes with a zero high limb, and the protocol stores their full ring sum through `pushOpenedValue`. Only the returned output is reduced modulo B. Thus the stored opening and checked aggregate are

$$o_i = \sum_j \ell(s_{ij}) \pmod M, \qquad y = \sum_i r_i o_i \pmod M.$$

In particular, o_i retains carries from summing low shares. It is not the low-only application output.

In `Spdz2kMacCheckComputation.computeDifference`, the high limb of $\ell(s) - s$ is $-h(s)$ mod B. Accordingly, `computePValues` broadcasts

$$p_j = -\sum_i r_i h(s_{ij}) + \ell(q_j) \pmod B, \qquad p = \sum_j p_j \pmod B.$$

`CompUIntConverter128.createFromHigh` embeds p as a low-valued `CompUInt128`. `CompUInt128.shiftLowIntoHigh` subsequently multiplies a value by B modulo M. Hence `computeZValues` commits to residuals whose sum is

$$z_j = \alpha_j y - \sum_i r_i \gamma_{ij} - Bp\alpha_j + B\eta_j,$$
$$\zeta = \sum_j z_j = \alpha(y - Bp) - \sum_i r_i \sum_j \gamma_{ij} + B \sum_j \eta_j \pmod M.$$

These expressions include the source's low-product shifts: multiplying by B discards the product's high limb modulo B^2.

Now define the actual modified shares by $s'_{ic} = (s_{ic} + e_i) \bmod M$. Set

$$\Delta\ell_i = \ell(s'_{ic}) - \ell(s_{ic}), \qquad \Delta h_i = h(s'_{ic}) - h(s_{ic}).$$

These are differences of the before/after limbs, rather than a decomposition of e_i itself. They satisfy $e_i \equiv \Delta\ell_i + B\Delta h_i \pmod M$. For example, when $s_{ic} = B - 1$ and $e_i = 1$, the differences are $1 - B$ and 1. Thus the carry is included explicitly. The source formulas give

$$\Delta y = \sum_i r_i \Delta\ell_i \pmod M, \qquad \Delta p = -\sum_i r_i \Delta h_i \pmod B.$$

The coupled MAC and mask terms cancel between executions, yielding

$$\begin{aligned}\Delta\zeta &= \zeta_{\text{modified}} - \zeta_{\text{honest}} \\ &= \alpha(\Delta y - B\Delta p) \\ &= \alpha \sum_i r_i(\Delta\ell_i + B\Delta h_i) = \alpha \sum_i r_i e_i \pmod M.\end{aligned}$$

For a zero-residual baseline, $\sum_i r_i e_i = 0$ therefore preserves zero residual. The vector $e_i = r_j$, $e_j = -r_i$ satisfies this condition without an inverse; when $0 < r_i, r_j < B$, both output low parts change. The final test in `buildComputation` accepts exactly a zero sum of opened residuals after its preceding stages complete. This derivation covers the coupled share-replacement path, rather than arbitrary malicious correction messages or complete protocol security.

A.2 One opening and the key-distribution premise

Retain the coupling conditions above and a zero-residual baseline. With one coefficient $r \in \{0, \ldots, B-1\}$, zero residual requires $2^{128} \mid \alpha r e$. For $r \neq 0$, write $v = v_2(r) \leq 63$. If α is odd, acceptance requires $2^{128-v} \mid e$, so the opened low 64 bits cannot change. For nonzero even α, put $a = v_2(\alpha)$. A low-bit-changing shift has $w = v_2(e) \leq 63$ and requires $a + v + w \geq 128$, hence $a + v > 64$. Either $r = 0$ or $\alpha = 0$ gives zero residual for every shift.

The fixed dummy harness passes `factory.createRandom()` as each party's key share. `CompUInt128Factory.createRandom` samples sixteen bytes, and `Spdz2kDummyDataSupplier` retains the full 128-bit share for MAC multiplication [16]. This identifies the key representation and sampling call. Neither the constructor nor dummy execution establishes that the aggregate key remains uniform given the adversary's complete view.

For the probability calculation, explicitly assume that, conditional on the pre-attempt view and the selected (r, e), α remains uniform over $\mathbb{Z}_{2^{128}}$. For nonzero r and a low-bit-changing e,

$$\Pr[\alpha r e \equiv 0 \pmod{2^{128}} \mid \mathsf{view}, r, e] = 2^{-(128-v-w)} \leq 2^{-(65-v)}.$$

The bound reaches $1/4$ at $r = 2^{63}$ and $w = 63$. For a single coefficient additionally uniform on all 2^{64} values and independent of the key, the nonzero-coefficient contribution is at most

$$\sum_{v=0}^{63} 2^{-(v+1)} 2^{-(65-v)} = 64 \cdot 2^{-66} = 2^{-60}.$$

The zero coefficient adds 2^{-64}, giving

$$\Pr[\text{accepted low-bit-changing shift}] \leq 2^{-60} + 2^{-64} = 17 \cdot 2^{-64}.$$

Conditional on $r \neq 0$, the average bound is $2^{-60}/(1 - 2^{-64})$. The zero-key event is already included and adds no separate term. The adversary may choose e after seeing r, subject to the

conditional-key premise. These are single-attempt bounds under the stated distributions; they do not cover selected favorable batches, repeated attempts, concurrent sessions, or reused keys with an accumulated adversarial view. The recorded high-bit-only edits change no output low bits and supply no single-opening output-corruption evidence.

B Minimal reproduction recipe for the core experiment

B.1 Environment and scope

This recipe uses the present `05-challenge-binding/` layout and a separate checkout of FRESCO commit `f8383e6aead6b44d44b798553aaf0bcaccd81dab`. It covers the message-level opening experiment and the recorded candidate-patch comparison. It does not invoke the broader `run_all.py` harness or depend on its missing historical source snapshots.

The retained message-level report identifies Eclipse Temurin OpenJDK `21.0.11+10-LTS` on Windows 11 amd64. The command printed in `code/artifact/kernel/kernel/messagel evel/run.log` invokes a distribution directory named `apache-maven-3.9.9`; an independent Maven version banner was not retained. Use JDK 21 and Maven 3.9.9 to match this recorded environment, and retain the actual version output for a new run. The pinned parent POM sets Java source/target 1.8, but `lib/common/pom.xml` overrides them to 10; Java 8 alone is therefore insufficient for this reactor. The inspected POMs do not enforce a Maven minimum. These are build declarations and historical environment records, not a newly measured compatibility matrix. Pinned parent POM, common-module POM.

The commands below are for PowerShell, with Git, JDK and Maven on `PATH`, starting in the directory containing `05-challenge-binding/`. They create a uniquely named reproduction directory and an isolated Maven repository. Initial dependency retrieval requires network access. The core commands were executed during the clean-checkout verification described in Section 6.2. That run used the downloaded Maven executable and an explicit isolated review directory in place of the PATH lookup and GUID-named directory below; the message-level and patch-comparison blocks were extracted from the manuscript without changing the test code or POMs. Its inputs and results are retained in `data/verification-2026-11-03/`.

B.2 Pinned checkout and dependency build

```
$packageRoot = (Resolve-Path -LiteralPath './05-challenge-binding').Path
$pin = 'f8383e6aead6b44d44b798553aaf0bcaccd81dab'
$reproRoot = Join-Path (Get-Location).Path ('fresco-repro-' + [guid]::NewGuid().ToString('N'))
New-Item -ItemType Directory -Path $reproRoot -ErrorAction Stop | Out-Null
$frescoRoot = Join-Path $reproRoot 'fresco'
$mavenCache = Join-Path $reproRoot 'm2'

git clone --no-checkout https://github.com/aicis/fresco.git $frescoRoot
if ($LASTEXITCODE -ne 0) { throw 'Clone failed' }
git -C $frescoRoot checkout --detach $pin
if ($LASTEXITCODE -ne 0) { throw 'Pinned checkout failed' }
$actualPin = (git -C $frescoRoot rev-parse HEAD).Trim()
if ($LASTEXITCODE -ne 0 -or $actualPin -ne $pin) { throw 'Revision mismatch' }

$mvnExe = (Get-Command mvn.cmd -ErrorAction Stop).Source
& $mvnExe --version 2>&1 | Tee-Object -FilePath (Join-Path $reproRoot 'maven-version.log')
if ($LASTEXITCODE -ne 0) { throw 'Cannot query Maven version' }
java -version 2>&1 | Tee-Object -FilePath (Join-Path $reproRoot 'java-version.log')
if ($LASTEXITCODE -ne 0) { throw 'Cannot query Java version' }

Set-Location -LiteralPath $frescoRoot
$mavenArgs = @('-B', '-ntp', "-Dmaven.repo.local=$mavenCache")
& $mvnExe @mavenArgs -pl suite/spdz2k -am -DskipTests install 2>&1 |
    Tee-Object -FilePath (Join-Path $reproRoot 'bootstrap.log')
if ($LASTEXITCODE -ne 0) { throw 'Dependency build failed; inspect bootstrap.log' }
```

`-am` builds the selected module's reactor dependencies. The SPDZ2k POM depends on `core`, `common`, and `commitment`, including the `core` and `common` test JARs. Those test utilities are needed by the probe. `-DskipTests` skips execution while preserving test compilation and test-JAR packaging; replacing it with `-Dmaven.test.skip=true` can remove required test artifacts. The installation is to the separate local repository; no deployment or publishing goal is used. Pinned SPDZ2k POM, Maven reactor options, Maven test-skipping semantics.

B.3 Message-level probe and result markers

```
$kernelRoot = Join-Path $packageRoot 'code/artifact/kernel/kernel'
$testPackage = 'dk/alexandra/fresco/suite/spdz2k/protocols/computations'
$testDir = Join-Path $frescoRoot ('suite/spdz2k/src/test/java/' + $testPackage)
New-Item -ItemType Directory -Path $testDir -Force -ErrorAction Stop | Out-Null
$messageTest = Join-Path $kernelRoot 'messagelevel/TestKernelMessageLevel.java'
Copy-Item -LiteralPath $messageTest -Destination $testDir -ErrorAction Stop
Get-FileHash -Algorithm SHA256 -LiteralPath $messageTest |
    Format-List | Out-File -FilePath (Join-Path $reproRoot 'test-source-hash.txt')

& $mvnExe @mavenArgs -pl suite/spdz2k '-Dtest=TestKernelMessageLevel' test 2>&1 |
    Tee-Object -FilePath (Join-Path $reproRoot 'message-level.log')
$messageExit = $LASTEXITCODE
Select-String -LiteralPath (Join-Path $reproRoot 'message-level.log') `
    -Pattern '^MSGLVL_(HONEST|TAMPER|CONTROL)_RESULT'
if ($messageExit -ne 0) { throw 'Message-level test command failed; inspect its log' }

$resultText = Get-Content -LiteralPath (Join-Path $reproRoot 'message-level.log') -Raw
if ($resultText -match '(?m)^MSGLVL_(HONEST|TAMPER)_RESULT (blocked|error)') {
    throw 'An honest or tampered execution did not complete normally'
}
if ($resultText -notmatch 'MSGLVL_HONEST_RESULT status=0 opened=\[50, 600\]' -or
    $resultText -notmatch 'MSGLVL_CONTROL_RESULT blocked') {
    throw 'Required honest or negative-control evidence is absent'
}
$tamperMatches = [regex]::Matches($resultText,
    'MSGLVL_TAMPER_RESULT status=1 opened=\[([0-9]+),\s*([0-9]+)\]')
if ($tamperMatches.Count -ne 3) { throw 'Expected three normal tampered executions' }
$outputModulus = [System.Numerics.BigInteger]::Parse('18446744073709551616')
foreach ($tamperMatch in $tamperMatches) {
    $observedSum = [System.Numerics.BigInteger]::Parse($tamperMatch.Groups[1].Value)
    $observedProduct = [System.Numerics.BigInteger]::Parse($tamperMatch.Groups[2].Value)
    $functionalResidual = (($observedProduct - 20 * ($observedSum - 20)) %
        $outputModulus + $outputModulus) % $outputModulus
    if ($functionalResidual -eq 0) { throw 'This pair does not demonstrate the claimed violation' }
    Write-Output "Nonzero functional residual: $functionalResidual"
}
```

The current packaged class has five test methods: an honest run, three-party tampering, two- and four-party tampering, and a three-party non-kernel control. Each method initializes its own fresh joint randomness. The original initial log contains three methods, whereas the later multiparty record contains five; this code/layout recipe targets the packaged five-method class.

The expected evidence is a normal `MSGLVL_HONEST_RESULT status=0 opened=[50, 600]`, normal `MSGLVL_TAMPER_RESULT status=1 opened=[S, P]` lines with changed outputs, and `MSGLVL_CONTROL_RESULT blocked`. The tampered values vary with the seed; `status=1` is a test tamper flag, not an exit code. A normal result line is printed only after `runTest` returns. A tampered `blocked` or `error` line is not an accepted forgery, even if the existing test assertions happen to pass. Retain the entire log and Maven exit code, and require no such tampered failure line. For the functional-integrity interpretation, also evaluate the relation in Section 4: with honest input 20, any lawful corrupt input requires $P - 20(S - 20) = 0 \pmod{2^{64}}$. The retained main tampered result violates it.

The test uses dummy preprocessing and a shared static result slot; it does not preserve a separate final result for each honest party. Its source and normal-return markers support

the limited message-level claim described in Section 4. Its stream-replay helper requests four coefficients because it accounts for two multiplication openings followed by two final-output openings; this must not be read as a claim that the four openings form one MAC-check batch.

B.4 Candidate patch comparison

Copy the old disclosed suite to the same test package and first run it against the unmodified pinned source. This provides a local pre-patch record; the oracle limitations identified in Section 4 still apply.

```
$disclosedTest = Join-Path $kernelRoot 'disclosed/TestSpdz2kMacCheckCancellation.java'
Copy-Item -LiteralPath $disclosedTest -Destination $testDir -ErrorAction Stop
& $mvnExe @mavenArgs -pl suite/spdz2k '-Dtest=TestSpdz2kMacCheckCancellation' test 2>&1 |
    Tee-Object -FilePath (Join-Path $reproRoot 'disclosed-original.log')
if ($LASTEXITCODE -ne 0) { throw 'Original-suite command failed; inspect its log' }

$patchFile = Join-Path $kernelRoot 'disclosed/fix.patch'
git apply --check $patchFile
if ($LASTEXITCODE -ne 0) { throw 'Patch does not apply to this checkout' }
git apply $patchFile
if ($LASTEXITCODE -ne 0) { throw 'Patch application failed' }
git diff --name-only | Tee-Object -FilePath (Join-Path $reproRoot 'patched-files.txt')

& $mvnExe @mavenArgs -pl suite/spdz2k '-Dtest=TestSpdz2kMacCheckCancellation' test 2>&1 |
    Tee-Object -FilePath (Join-Path $reproRoot 'disclosed-patched.log')
$patchedExit = $LASTEXITCODE
Write-Output "Patched-suite exit code: $patchedExit"
$reportName = 'dk.alexandra.fresco.suite.spdz2k.protocols.computations.' +
    'TestSpdz2kMacCheckCancellation.txt'
$reportPath = Join-Path $frescoRoot ('suite/spdz2k/target/surefire-reports/' + $reportName)
Select-String -LiteralPath $reportPath -Pattern 'Tests run:|Mac check failed'
```

The patch changes four production files: `Spdz2kMacCheckComputation.java`, `Spdz2kResourcePool.java`, `Spdz2kResourcePoolImpl.java`, and `Spdz2kRoundSynchronization.java`. It derives per-check randomness through `CoinTossingComputation` and the existing commitment machinery. Test copies are additional untracked inputs; the patch does not alter their assertions or the POMs.

The retained patched-suite outcome is eleven tests, zero assertion failures, seven errors with `MaliciousException: Mac check failed`, and four completed honest baselines. A nonzero Maven exit is expected for that historical outcome because the old attack tests still expect acceptance. It is not, by itself, mitigation evidence: distinguish these seven MAC-check rejections from compilation failures, missing dependencies, or unrelated exceptions. The historical text report is `code/artifact/kernel/kernel/disclosed/dk.alexandra.fresco.suite.spdz2k.protocols.computations.TestSpdz2kMacCheckCancellation.txt`. Use it for the reported result. The adjacent historical sanitized XML contains unescaped path placeholders and cannot be assumed machine-parseable. The valid XML reports from the dated clean-checkout verification are separately retained in `data/verification-2026-11-03/reports/`. Preserve newly generated logs/reports as a new execution record rather than replacing the retained evidence. The patch remains a candidate mitigation with the security limits in Section 4.

B.5 Scope of the auxiliary tool reports

The `binding.py` report demonstrates the recorded nonzero two-mask field instance, masks `[45747, 6372]` modulo 65519 and witness `[6372, 19772]`. The present implementation is not a complete binding decision procedure for arbitrary supplied masks or power-of-two rings: zero-mask cases and one-position zero-divisor cases are not handled correctly. Accordingly, no generic soundness, completeness, or single-position binding claim is inferred from it.

The synthesis reports establish their explicitly checked weighted residual and low-part changes. Their field named `kernel_multiplicity_at_a` returns the positive kernel period $M/\gcd(r_a, M)$ for the single-coefficient multiplication map, not its kernel cardinality $\gcd(r_a, M)$. When the coefficient is a unit, the reported period is M, representing the zero residue rather than a nonzero kernel element. This field is not used as a multiplicity or an attack-probability estimate in the manuscript.

C Descriptive audit material

C.1 Scope across check families

C1 and C2 identify inspection points whose meaning depends on the check. Table C1 separates the mechanisms covered by the tools from those used only as context.

Table C1: Inspection targets and limits.

Family	**Role of the challenge or digest**	**Inspection target**	**Protocol-specific requirement**	**Coverage here**
Linear compression	weights for a batch	coefficient provenance and admissible errors	arithmetic domain, hidden state, and correction terms	OrderTool candidates and FRESCO case study
Polynomial identity	evaluation point	committed coefficients and point derivation	degree and field assumptions	contextual discussion
Lookup or permutation	relation randomization	table and witness binding	binding may be supplied by other commitments	contextual discussion
Fiat-Shamir transcript	derived challenge	pre-challenge fields, encoding, and domain context	security of the complete proof protocol	annotated monitor
Hash consistency	equality digest	input encoding and commitment order	distinct collision, preimage, and privacy arguments	manual scope discussion

A deterministic digest comparison does not acquire a fixed-message sampling bound merely by appearing in a consistency check. Likewise, leakage before an eventual abort raises a privacy question beyond the integrity predicates in Section 2. The audit vocabulary identifies questions to investigate; it supplies no common security theorem for these families.

C.2 Audit aids and their validation

C.2.1 Annotated transcript monitor

The transcript monitor consumes manually constructed specifications with events such as `absorb`, `squeeze`, and `check`. At a checked challenge, it compares an annotated required set with the dependencies accumulated by the transcript. Its executable rule is set containment, with explicit annotations for supported alternatives such as constraint-level binding. It also records whether a domain value has been supplied.

The rule evaluates the supplied specification. Source inspection is needed to justify the required set, encoding assumptions, and annotations. The monitor does not establish encoding injectivity, complete source-program semantics, or domain separation across protocol contexts. A correct

set-containment calculation can therefore coexist with an incorrect transcription or an incomplete required set.

The artifact contains 26 specifications, including repaired configurations, comparison variants, and synthetic fixtures. Sixteen of the 19 entries in Appendix C.3 have corresponding specifications; Plonky2 and SP1 are source-inspection entries, and Nova is outside the modeled check. A separate evidence script declares 14 source-pattern assertions in total. These counts describe regression inputs and selected source checks, rather than independent security audits. Section 6.2 distinguishes their retained results from the checks repeated for this revision.

C.2.2 OrderTool

OrderTool selects check-like functions from brace-delimited source code and records approximate randomness provenance and message order. Its heuristics recognize public-looking generator pools, secret-shared randomizer patterns, locally created randomness, and hash-based code. A limited traversal expands some callees. The output is a list of candidates for manual inspection.

The boundary proxy is a recognized send, receive, or opening operation. It may differ from the point at which a protocol actually binds the adversary's choices. Name matching cannot establish that randomness remains secret or that an earlier seed is unpredictable. Call-expansion order can affect a classification, and block-comment removal can shift reported line numbers. Each candidate must therefore be checked in the original source.

The analysis follows this sequence:

```
source files -> check-like functions -> provenance and message-pattern heuristics
             -> candidate records -> manual source and protocol analysis
```

Across the ten selected targets in Table C3, the retained reports contain 347 records, including constructors, accessors, tests, and protocol routines. Their labels comprise 29 `post-message`, one `pre-existing`, one `pre-message`, 35 `hash-based`, and 281 `different-mechanism` records. The labels encode heuristic rules: `pre-existing` flags an apparent public pool or generator; `pre-message` flags a recognized local-randomness operation before the first recognized message; and `post-message` also includes patterns treated as fresh or secret-shared. `hash-based` identifies a hash or commitment pattern without recognized coefficients, while `different-mechanism` records that the analyzer found no recognized randomness. None of these labels establishes the mechanism's security or even the absence of an unrecognized randomized check.

The retained mutation study changes randomness at six sites. Four changes are detected and two are missed by function selection; the associated reports are under `data/ordertool/out/mutation-*`. These selected cases illustrate tool behavior without estimating population precision or recall. Classification rules were developed using some of the repositories that supply evaluation cases, which further limits generalization.

C.2.3 Interpreting the outputs

The transcript monitor checks an author-supplied dependency model; OrderTool extracts source-pattern candidates. Their outputs have different meanings and cannot be combined into one security verdict. The narrower coefficient-source audit in `data/synth/audit-report.json` records three per-file classifications for FRESCO: `Spdz2kMacCheckComputation` is a candidate, while `SpdzMacCheckProtocol` and `SpdzRoundSynchronization` are classified as fresh. These are also heuristic classifications.

The companion manuscript's manual audit of 17 checks in eight frameworks is a separate dataset. Those counts are excluded from the totals here. The present paper reports the ten selected scan targets and the evidence for the FRESCO case study.

C.3 Descriptive corpus

C.3.1 Selection and counting units

The zero-knowledge corpus reuses the project's earlier transcript investigation. Table C2 has 19 implementation entries spanning 18 projects because the gnark backend and recursion library are separate entries. Three rows describe historical vulnerable configurations. The 26 regression specifications include additional repaired configurations, comparison variants, and synthetic fixtures. The corpus provides context for C1 and is not an independent new survey.

The MPC corpus consists of ten selected scan targets. Selection depended on accessible consistency-check code and parser compatibility; some targets are subdirectories or protocol-specific slices. No complete population inventory or consistent exclusion log is available. MOTION and libscapi remain targets even though their records do not establish that the scanned code contains an SPDZ-style randomized MAC check.

The two investigations also differ in human effort. Transcript specifications require source reading and annotation; a source scan produces candidates that require further analysis. No contemporaneous time log supports a person-day comparison or a claim that the two corpora received equally deep audits.

C.3.2 Zero-knowledge implementation entries

Table C2: Transcript investigation entries.
Labels describe recorded coverage under a model or source inspection. Historical failures concern pre-fix configurations. Domain warnings are separate from coverage.

Entry	Implementation	Language	Coverage record	Evidence or qualification
1	snarkjs Plonk [38]	JS	pass	specification; domain warning
2	gnark backend [18]	Go	pass	specification; domain warning
3	gnark recursion [18]	Go	pass	specification; domain warning
4	FastCrypto KCP [14]	Rust	historical fail	pre-fix configuration
5	CIRCL qndleq [7]	Go	historical fail	pre-fix configuration
6	libsignal [24]	Rust	historical fail	pre-fix configuration
7	Halo2 [19]	Rust	pass	specification
8	Plonky2 [33]	Rust	pass	source inspection
9	Nova [30]	Rust	outside model	no monitor verdict
10	Bulletproofs [4]	Rust	pass	specification
11	arkworks Marlin [25]	Rust	pass	specification
12	Plonky3 uni-stark [34]	Rust	partial	modeled path only
13	SP1 [39]	Rust	pass	source inspection
14	Winterfell [46]	Rust	pass	specification
15	Kimchi [22]	Rust	pass	specification
16	Barretenberg [2]	C++	pass	specification
17	dusk-plonk [9]	Rust	pass	specification
18	jellyfish [21]	Rust	pass	specification
19	CL signatures [8]	Rust	pass	specification

Historical and repaired configurations provide regression examples. A passing model establishes coverage of its annotated required values, subject to those annotations. It does not rule out an omitted binding path or another vulnerability in the implementation.

C.3.3 MPC scan targets

Table C3: Selected MPC scan targets and retained parser records.
Records are check-like functions; candidates require manual investigation. A zero candidate count gives no repository-wide security conclusion.

Scan target	Language	Records	Candidates
MP-SPDZ [29]	C++	139	0
SCALE-MAMBA [37]	C++	63	0
OpenCoral [32]	C++	42	0
MPC4j ABB3 [28]	Java	30	0
MOTION [27]	C++	25	0
libscapi [23]	C++	19	0
FRESCO SPDZ2k [16]	Java	10	1
ark-mpc [1]	Rust	10	0
emp-agmpc [10]	C++	7	1
SPU SWIFT [42]	C++	2	0

The two candidates have different evidentiary outcomes. FRESCO has recorded accepted incorrect openings, examined in Section 4. The emp-agmpc observation concerns an isolated compression relation and remains unresolved at protocol level.

C.3.4 The emp-agmpc boundary case

The examined `check1` routine in emp-agmpc [11] compresses MAC-related arrays into 40 buckets using public randomness. A model with 40 rows, $L = 1024$, and rank 40 admits a nonzero kernel vector that avoids the tail examined by `check2`. This is a calculation over the modeled linear map; no emp-agmpc execution is reported.

The examined online use of keys and values does not directly turn this MAC-side perturbation into a computation error. Establishing a protocol attack would require a reachable malicious execution that changes a security-relevant result while passing the remaining checks. The retained result is therefore an unresolved check-level observation.

References

[1] Renegade. *ark-mpc*. `https://github.com/renegade-fi/ark-mpc`, 2023.

[2] Aztec. *Barretenberg*. `https://github.com/AztecProtocol/barretenberg`, 2020.

[3] Bernhard, David; Pereira, Olivier; Warinschi, Bogdan. *How not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios*. ASIACRYPT 2012, pp. 626–643. doi:10.1007/978-3-642-34961-4_38.

[4] Bünz, Benedikt; Bootle, Jonathan; Boneh, Dan; Poelstra, Andrew; Wuille, Pieter; Maxwell, Greg. *Bulletproofs: Short Proofs for Confidential Transactions and More*. IEEE S&P 2018, pp. 315–334. doi:10.1109/SP.2018.00020.

[5] Canetti, Ran; Chen, Yilei; Holmgren, Justin; Lombardi, Alex; Rothblum, Guy N.; Rothblum, Ron D.; Wichs, Daniel. *Fiat-Shamir: From Practice to Theory*. STOC 2019, pp. 1082–1090. doi:10.1145/3313276.3316380.

[6] Kolozyan, Arman; Vandenbogaerde, Bram; Swalens, Janwillem; Hoste, Lode; Chaliasos, Stefanos; De Roover, Coen. *Language-Agnostic Detection of Computation-Constraint Inconsistencies in ZKP Programs via Value Inference*. Cryptology ePrint Archive, Paper 2025/2120; IEEE S&P 2026. `https://eprint.iacr.org/2025/2120`.

[7] Cloudflare. *CIRCL: Cloudflare Interoperable Reusable Cryptographic Library*. `https://github.com/cloudflare/circl`, 2019.

[8] Hyperledger AnonCreds. *CL Signatures*. `https://github.com/hyperledger/anoncreds-clsignatures-rs`, 2022.

[9] Dusk Network. *dusk-plonk*. `https://github.com/dusk-network/plonk`, 2019.

[10] EMP Toolkit. *emp-agmpc*. `https://github.com/emp-toolkit/emp-agmpc`, 2022.

[11] Wang, Xiao; Ranellucci, Samuel; Katz, Jonathan. *Global-Scale Secure Multiparty Computation*. ACM CCS 2017, pp. 39–56. doi:10.1145/3133956.3133979.

[12] Bruggemann, Andreas; Schneider, Thomas. *When Trying to Catch Cheaters Breaks the MPC: Breaking and Fixing Delayed Consistency Checks in Trident, Fantastic Four, SWIFT, and Quad*. EUROCRYPT 2026, pp. 457–487. doi:10.1007/978-3-032-25324-8_16. `https://eprint.iacr.org/2026/234`.

[13] Dalskov, Anders; Escudero, Daniel; Keller, Marcel. *Fantastic Four: Honest-Majority Four-Party Secure Computation With Malicious Security*. USENIX Security 2021, pp. 2183–2200.

[14] Mysten Labs. *FastCrypto*. `https://github.com/MystenLabs/fastcrypto`, 2022.

[15] Fiat, Amos; Shamir, Adi. *How to Prove Yourself: Practical Solutions to Identification and Signature Problems*. CRYPTO '86, pp. 186–194. doi:10.1007/3-540-47721-7_12.

[16] Alexandra Institute. *FRESCO: A Framework for Efficient Secure Computation*. `https://github.com/aicis/fresco/tree/f8383e6aead6b44d44b798553aaf0bcaccd81dab`, 2026. Revision f8383e6aead6b44d44b798553aaf0bcaccd81dab; pinned source inspected 2026-11-03.

[17] Miller, Jim. *Coordinated disclosure of vulnerabilities affecting Girault, Bulletproofs, and PlonK*. Trail of Bits Blog, April 2022. `https://blog.trailofbits.com/2022/04/13/part-1-coordinated-disclosure-of-vulnerabilities-affecting-girault-bulletproofs-and-plonk/`.

[18] Consensys. *gnark*. `https://github.com/Consensys/gnark`, 2021.

[19] Zcash Foundation. *Halo2*. `https://github.com/zcash/halo2`, 2023.

[20] FRESCO developers. *Bug in MAC check for SPDZ-based protocols*. GitHub issue #324, `https://github.com/aicis/fresco/issues/324`, 2018. Opened 2018-12-10; still open as of 2026-10-20.

[21] Espresso Systems. *Jellyfish*. https://github.com/EspressoSystems/jellyfish, 2021.

[22] O(1) Labs. *Kimchi: Mina Proof System*. https://github.com/o1-labs/proof-systems, 2020.

[23] Bar-Ilan University. *libscapi*. https://github.com/cryptobiu/libscapi, 2016.

[24] Signal Foundation. *libsignal*. https://github.com/signalapp/libsignal, 2020.

[25] Chiesa, Alessandro; Hu, Yuncong; Maller, Mary; Mishra, Pratyush; Vesely, Noah; Ward, Nicholas. *Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS*. EUROCRYPT 2020, pp. 738–768. doi:10.1007/978-3-030-45721-1_26.

[26] Keller, Marcel; Orsini, Emmanuela; Scholl, Peter. *MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer*. ACM CCS 2016, pp. 830–842. doi:10.1145/2976749.2978357.

[27] TU Darmstadt. *MOTION: A Framework for Mixed-Protocol Multi-Party Computation*. https://github.com/encryptogroup/MOTION, 2020.

[28] Alibaba. *MPC4j*. https://github.com/alibaba-edu/mpc4j, 2022.

[29] Keller, Marcel. *MP-SPDZ: A Versatile Framework for Multi-Party Computation*. ACM CCS 2020, pp. 1575–1590. doi:10.1145/3372297.3417872.

[30] Kothapalli, Abhiram; Setty, Srinath; Tzialla, Ioanna. *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes*. CRYPTO 2022, pp. 359–388. doi:10.1007/978-3-031-15985-5_13.

[31] Keller, Marcel; Navot, Sela; Tyagi, Nirvan. *Off-Key: Inconsistent MAC Keys Compromise Privacy in SPDZ-Family MPC*. Cryptology ePrint Archive, Paper 2026/2533, 2026. https://eprint.iacr.org/2026/2533.

[32] Ant Group. *OpenCoral*. https://github.com/AntCPLab/OpenCoral, 2024.

[33] Polygon Zero. *Plonky2*. https://github.com/0xPolygonZero/plonky2, 2022.

[34] Polygon Zero. *Plonky3*. https://github.com/Plonky3/Plonky3, 2024.

[35] Pointcheval, David; Stern, Jacques. *Security Proofs for Signature Schemes*. EUROCRYPT 1996, pp. 387–398. doi:10.1007/3-540-68339-9_33.

[36] Kyster, Alexander; Nielsen, Frederik Huss; Oechsner, Sabine; Scholl, Peter. *Rushing at SPDZ: On the Practical Security of Malicious MPC Implementations*. IEEE S&P 2025, pp. 2491–2508. doi:10.1109/SP61157.2025.00176. https://eprint.iacr.org/2025/789.

[37] KU Leuven COSIC. *SCALE-MAMBA*. https://github.com/KULeuven-COSIC/SCALE-MAMBA, 2026.

[38] iden3. *snarkjs*. https://github.com/iden3/snarkjs, 2020.

[39] Succinct Labs. *SP1: A Performant, Open-Source zkVM*. https://github.com/succinctlabs/sp1, 2024.

[40] Damgard, Ivan; Pastro, Valerio; Smart, Nigel; Zakarias, Sarah. *Multiparty Computation from Somewhat Homomorphic Encryption*. CRYPTO 2012, pp. 643–662. doi:10.1007/978-3-642-32009-5_38.

[41] Cramer, Ronald; Damgard, Ivan; Escudero, Daniel; Scholl, Peter; Xing, Chaoping. *SPDZ2k: Efficient MPC mod 2^k for Dishonest Majority*. CRYPTO 2018, pp. 769–798. doi:10.1007/978-3-319-96881-0_26. https://eprint.iacr.org/2018/482.

[42] SecretFlow. *SPU: Secure Processing Unit*. https://github.com/secretflow/spu, 2023.

[43] Koti, Nishat; Pancholi, Mahak; Patra, Arpita; Suresh, Ajith. *SWIFT: Super-fast and Robust Privacy-Preserving Machine Learning*. USENIX Security 2021, pp. 2651–2668.

[44] Chaudhari, Harsh; Rachuri, Rahul; Suresh, Ajith. *Trident: Efficient 4PC Framework for Privacy Preserving Machine Learning*. NDSS 2020. doi:10.14722/ndss.2020.23005.

[45] Dao, Quang; Miller, Jim; Wright, Opal; Grubbs, Paul. *Weak Fiat-Shamir Attacks on Modern Proof Systems*. IEEE S&P 2023, pp. 199–216. doi:10.1109/SP46215.2023.10179408.

[46] Facebook Novi. *Winterfell: A STARK Prover and Verifier*. `https://github.com/facebook/winterfell`, 2019.

[47] Angel, Sebastian; Cohen, Sydnie-Shea; Ioannidis, Eleftherios; Mishra, Pratyush; Ozdemir, Alex; Qiu, Yuxiang; Shirzad, Alireza. *Zippel: A Language, Compiler, Runtime, and Analyzer for ZK Proof Systems*. Cryptology ePrint Archive, Paper 2026/2517; IEEE S&P 2027. `https://eprint.iacr.org/2026/2517`.